

Agentic AI Workflow for Defect Detection



Takuto Yoshida, Navya Bhat, Ayush Panigrahy, Roddy Taing





AGENDA

What we will walk through



01

Problem Setup

Octet lattices, hidden strut defects, and why CT alone is not enough.



02

Methodology

Segmentation, registration, and per-strut defect scoring.



03

Agentic Workflow

How the agent chains the steps and validates its own findings.



04

Dashboard & Demo

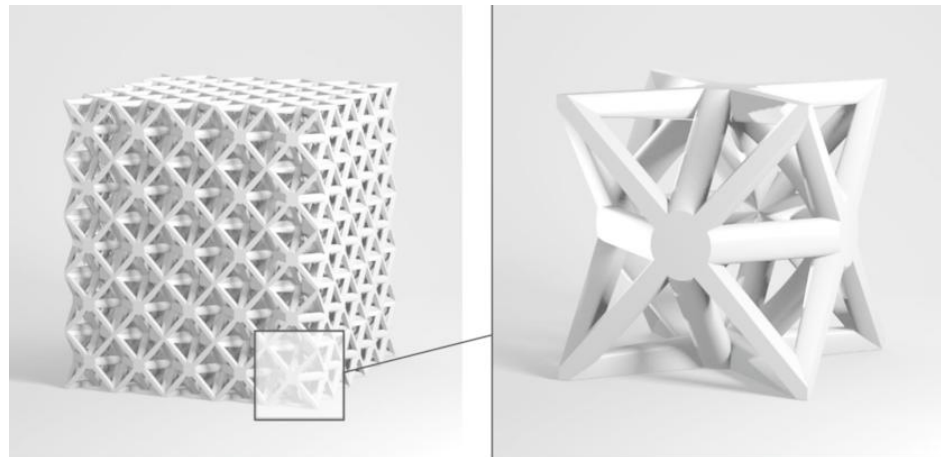
Reviewing ranked defect candidates slice by slice.



Problem Setup

Problem Context

- 1 **Octet lattice used in 3D printing**
Octet lattices are lightweight, load-bearing structures built from repeating octet unit cells, making them a common design in **additive manufacturing**.
- 2 **Defects hide in the geometry**
Their repeating 3D geometry makes internal defects difficult to inspect visually.
- 3 **Noisy volumetric scans**
Noisy volumetric scans blur strut boundaries and make reliable 3D defect analysis even harder.
- 4 **Automate it**
We need a repeatable way to find hidden defects and validate them across slices.



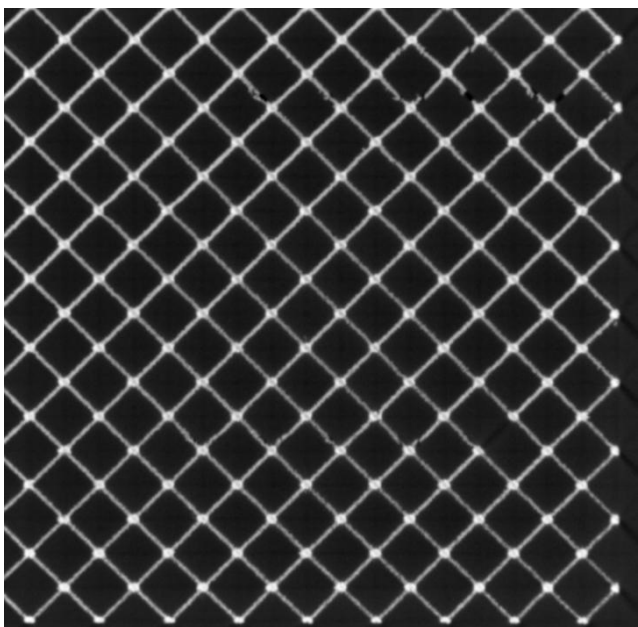
Octet-truss lattice → the unit cell magnified

A single unit cell repeats hundreds of times across the part; one absent strut looks like every other dark void.

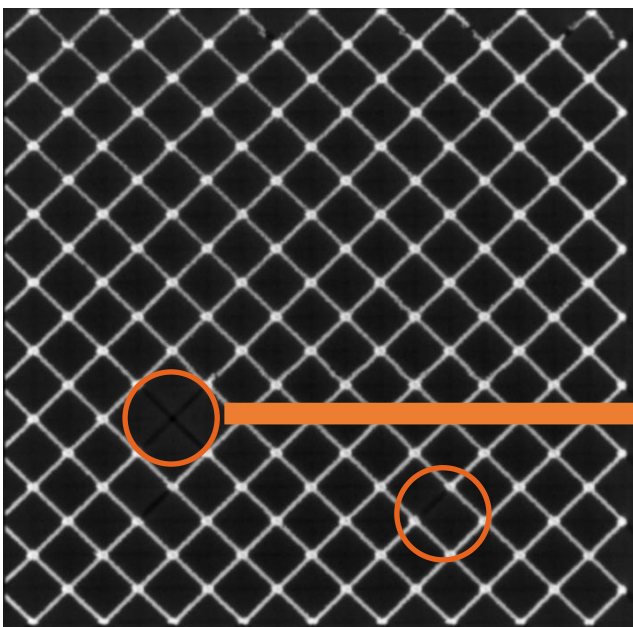
Source: reproduced from “The Scaled Boundary Finite Element Method for Dispersive Wave Propagation in Higher-Order Continua” (2022), ResearchGate.

Our goal: Detect missing and weak struts

- A strut is one of the thin bars that connects the lattice together and helps it carry load.
- The objective is to localize the defect, verify it across slices, and support downstream analysis

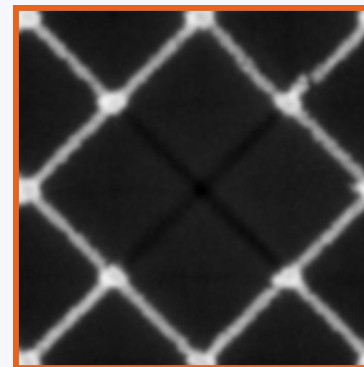


Nominal slice — all struts present



Defective slice — two missing struts

What the defect looks like up close

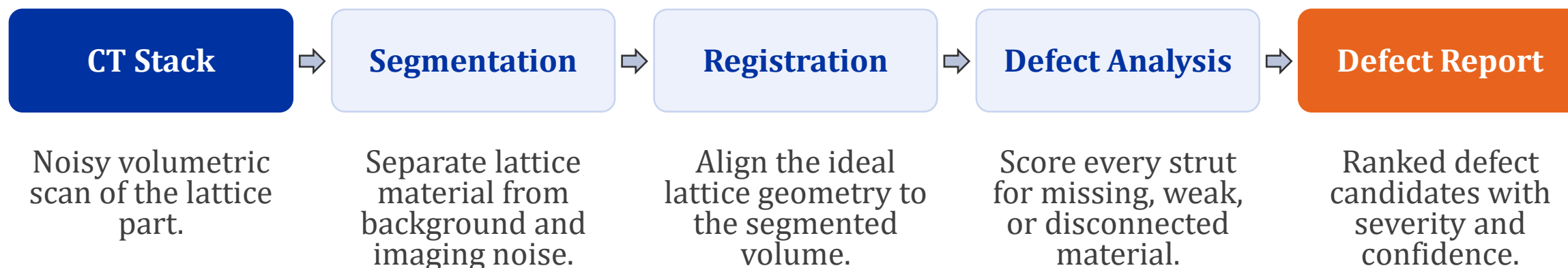


The four struts around this node are faint or absent, so the cell reads as a dark void: that signature is what the per-strut metrics are built to score.



Methodology

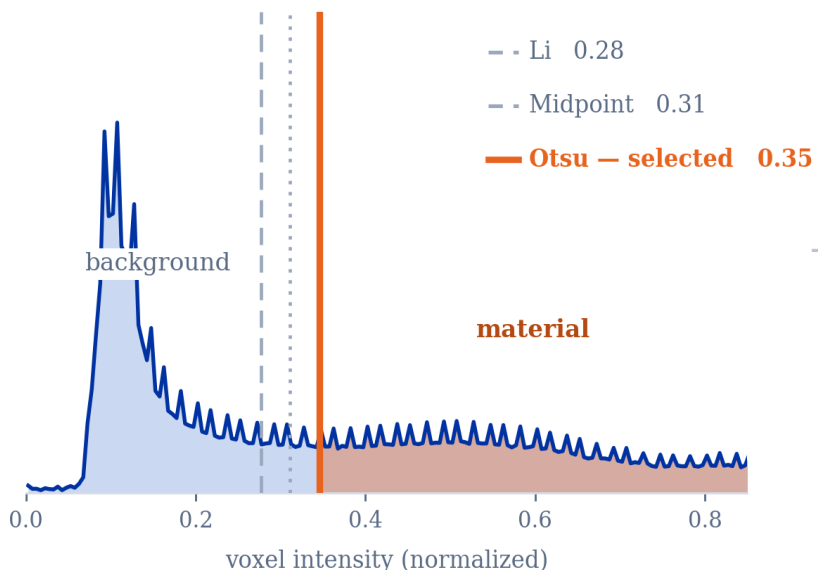
Overall Methodology: From CT stack to defect identification



This workflow converts raw volumetric CT data into registered structure and strut-level defect classifications

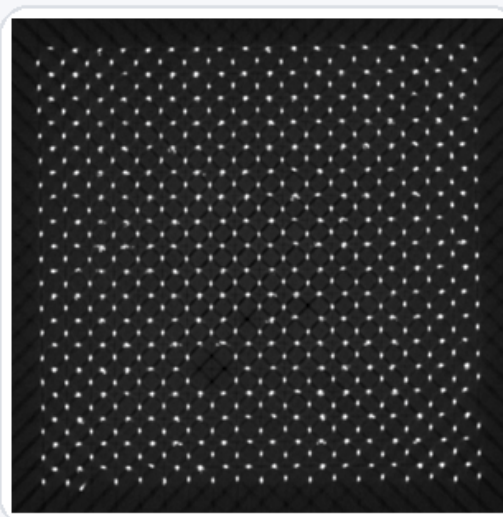
Segmentation using Otsu's thresholding

Threshold candidates on the sampled histogram

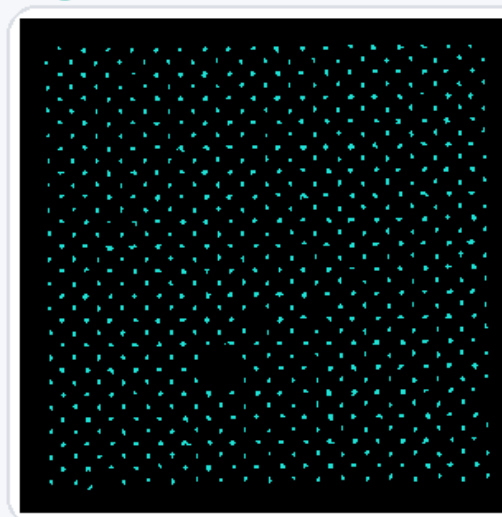


1% missing-strut specimen | current slice 365 of 765

Raw CT slice



Segmented mask



1 Sample the volume

We estimate one global intensity threshold from a coarse sample of the 3D CT volume rather than slice by slice.

2 Compare candidates

Otsu, Li, and their midpoint were all evaluated; Otsu produced the most plausible material fraction.

3 Binarize

The chosen threshold converts the volume into a binary mask that every downstream stage measures against.

Why Otsu: it picks the cut that splits the intensity histogram into two groups with the greatest possible variance between them.

Registration: aligning design to scan

Registration takes the ideal lattice geometry from the JSON file and aligns it to the segmented CT volume, so the expected struts match the observed structure.

INPUT

Ideal lattice geometry (JSON)

REFERENCE

Segmented CT volume / binary mask

OUTPUT

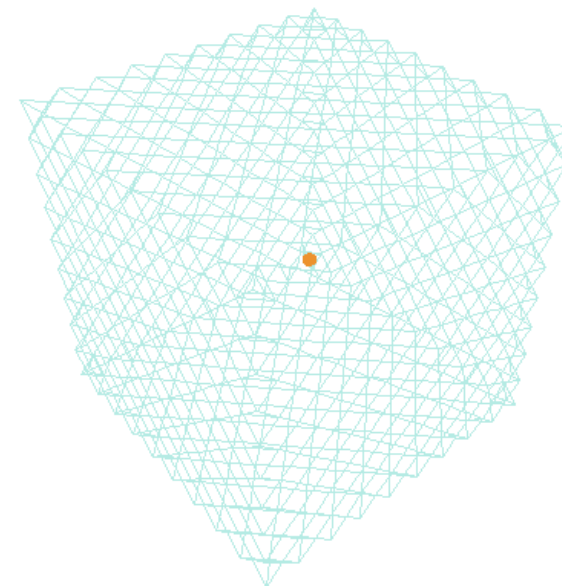
Registered lattice graph in scan coordinates

How we do it

- 1 **Bound the observed lattice:** Extract the lattice extent from the segmented mask to get the bounding volume.
- 2 **Fit scale, translation and rotations:** Isotropic scale and translation, and small z-axis rotation model space to voxel space.
- 3 **Apply the transform:** Generate the registered graph used downstream by defect analysis.

3D Registration from Design Octet Lattice to Scan Coordinates

1% missing-strut specimen



Representative visual for illustrative purposes only

Defect Analysis

How a strut is scored

- 1 Sample**
Walk the registered centerline of the strut point by point.
- 2 Measure**
Test each sample strut against the segmentation mask for material.
- 3 Score**
Roll the samples into four metrics, then a tier and a confidence.

Ranked by severity tier, then by confidence.

centerline_support

Fraction of sampled points along the centerline that land on material.

Low — the strut is barely there

gap_fraction

Longest unsupported run along the strut, as a fraction of its length.

High — a clean break in the middle

endpoint_min

Support at the weaker of the two strut ends.

Low — disconnected from its node

occupancy_ratio

Local material fraction in the neighbourhood around the strut.

Low — the whole region is empty



Turning metrics into a severity tier

TIER	TRIGGER CONDITION	WHAT IT MEANS	ACTION
Tier 3 Critical	centerline_support ≤ 0.25 , or gap_fraction ≥ 0.75 with occupancy_ratio ≤ 0.08	Strut is effectively absent	Flag first
Tier 2 Major	centerline_support ≤ 0.60 , or gap_fraction ≥ 0.50 , or endpoint_min ≤ 0.35	Substantial material missing or strut is broken	Review
Tier 1 Minor	centerline_support ≤ 0.82 , or gap_fraction ≥ 0.30 , or endpoint_min ≤ 0.65	Thin or partly supported	Monitor
Tier 0 Nominal	Everything else	Matches design intent	No action
Tier U Uncertain	Registration or volume-boundary uncertainty	Verdict not trustworthy	Re-check

Reading the rules: tiers are checked most-severe first; within a tier, candidates are ordered by confidence.



Agentic Workflow



Why agentify a pipeline that already runs?

"Agent-ifying" turns a brittle pipeline into a modular, rerunnable workflow with explicit state and stage-specific handoffs

	WITHOUT AGENTS — SCRIPT-BASED	WITH AGENTS
Enforce stage handoffs	Scripts write files, but nothing enforces a stage-to-stage contract.	Upstream agent publishes a handoff; downstream agent consumes it and publishes its own.
Resume at a missing stage	Control logic outside the stage scripts must detect missing stages and resume safely.	The orchestrator only calls subagents corresponding to missing artifacts.
Change segmentation logic from chat	External logic must track which outputs are stale and rerun the right scripts in order.	Chat asks segmentation agent to change method; registration and scoring rerun from the new mask.
Search for the best alignment	Alignment is one fixed transform, or a search you wire in and rerun as a whole.	Registration agent scores candidate alignments and publishes the winner with quality metrics.
Answer questions after a run	Read the logs and open the output files yourself.	Chat answers questions directly against the saved artifacts.
Change how a stage works	Changing behavior means editing and redeploying the script.	Update the skill and every agent that loads it follows the new procedure.



Our agentic workflow

1 Orchestrator Checks saved artifacts first, decides which stages already exist, and runs only what is missing, supporting both full reruns and partial refreshes.

2 Segmentation Agent

- Ingests the raw CT stack.
- Estimates a threshold to separate material from background.
- Publishes a binary mask with summary metrics and a QA slice.



3 Registration Agent

- Ingests the segmentation mask and nominal lattice graph.
- Searches candidate alignments.
- Publishes the registered graph with transform and alignment-quality metrics.



4 Defect Analysis Agent

- Ingests the registered graph and mask.
- Scores material support along each strut.
- Publishes candidate defect tables, severity tiers, and QA images.



5 Dashboard Agent

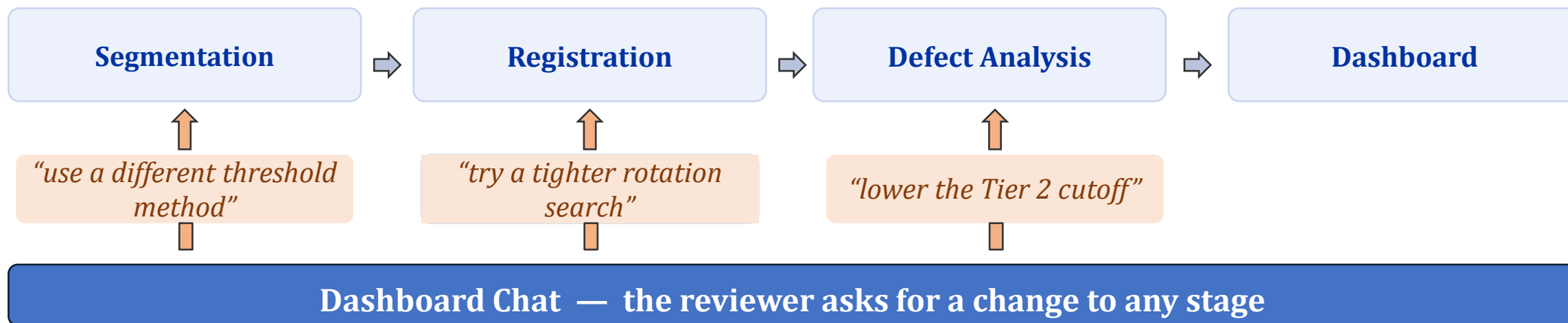
- Ingests saved workflow artifacts.
- Synthesizes them into review-ready summaries.
- Publishes the dashboard index and sample-level reports.

6 Dashboard chat agent Answer questions about segmentation, registration, or defect results and routes requested changes into controlled experimental runs.

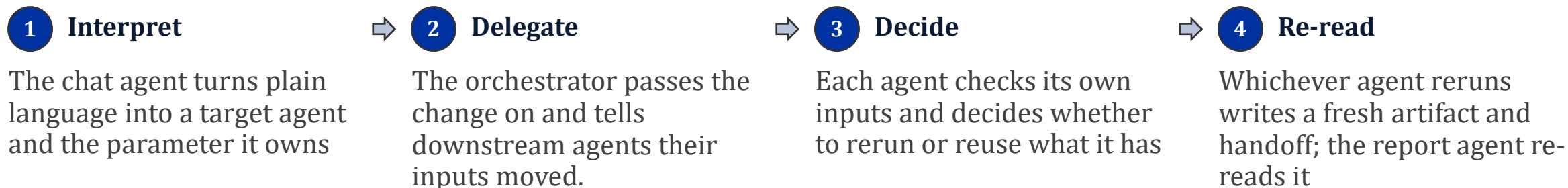
MCP: one unified framework

Human in the loop: chat as a control surface

The person reviewing the results is part of the workflow. Chat is where their judgement enters it : they change an upstream choice, and the change propagates through saved artifacts.



How agents handle that request





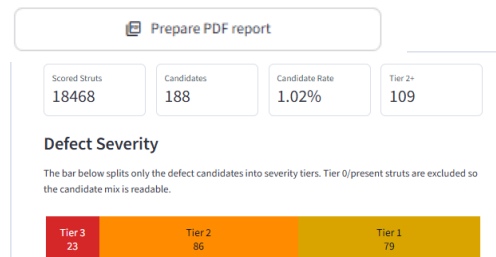
Dashboard and Demo



Dashboard Features

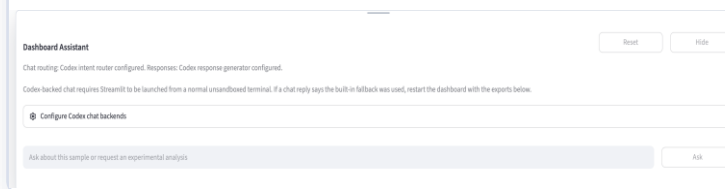
Report

Top-level triage: defect burden, severity mix, and “**export to pdf**” functionality



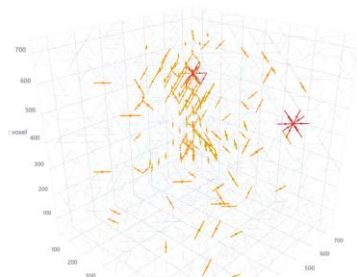
Chat Agent

Q&A about segmentation, registration, and candidates. Request ad hoc analyses



3D Inspector

Whole lattice in 3D with defect candidates overlaid



Strut Analysis

One strut at a time: metrics, CT overlay, and neighboring slices.



Linked: clicking a strut in the 3D Inspector loads that candidate straight into Strut Analysis.

TECH STACK

Streamlit

App shell, layout, tabs, chat UI

Three.js

Interactive 3D lattice inspector

Plotly

Charts and 3D fallback renderer

NumPy / scikit-image

Slice overlays and scientific figures

FastMCP / MCP

Workflow and assistant tool interface

ReportLab

PDF Export

Python

Application and orchestration layer

JSON / CSV / TIFF / PNG

Artifacts consumed by dashboard